

PointDog

Relatório Técnico — Sprint 4

Aplicativo do Prestador e Integração Final

1. Introdução

Esta sprint encerra o ciclo de desenvolvimento do PointDog com a entrega do aplicativo Flutter para o prestador de serviços e a integração completa do fluxo ponta a ponta: desde a solicitação criada pelo cliente até a conclusão pelo prestador, com comunicação assíncrona via Message-Oriented Middleware (MOM) e notificação em tempo real via WebSocket.

O sistema foi construído ao longo de quatro sprints incrementais, seguindo os princípios de Clean Architecture (MARTIN, 2019) e Event-Driven Architecture (EDA), com cada sprint adicionando uma camada funcional sobre a anterior:

- Sprint 1: Backend REST (Node.js/Express/Prisma/SQLite) com separação em camadas segundo Clean Architecture
- Sprint 2: Mensageria assíncrona com RabbitMQ (MOM/EDA), implementando os padrões Publish-Subscribe Channel e Topic Exchange descritos por Hohpe e Woolf (2003)
- Sprint 3: Aplicativo Flutter do cliente com integração REST, gerenciamento de estado com Provider e atualização em tempo real via WebSocket
- Sprint 4: Aplicativo Flutter do prestador integrado ao mesmo projeto, com roteamento WebSocket bidirecional e fluxo de ciclo de vida completo de agendamentos

A escolha de WebSocket para notificações em tempo real e RabbitMQ para processamento assíncrono segue os princípios de sistemas distribuídos estabelecidos por Coulouris et al. (2011): canais de comunicação indireta permitem desacoplar produtores de consumidores, aumentando a **escalabilidade** e a **resiliência** do sistema.

2. Arquitetura Implementada

2.1 Visão Geral do Sistema

O diagrama abaixo apresenta os três grandes componentes do sistema e seus meios de comunicação:

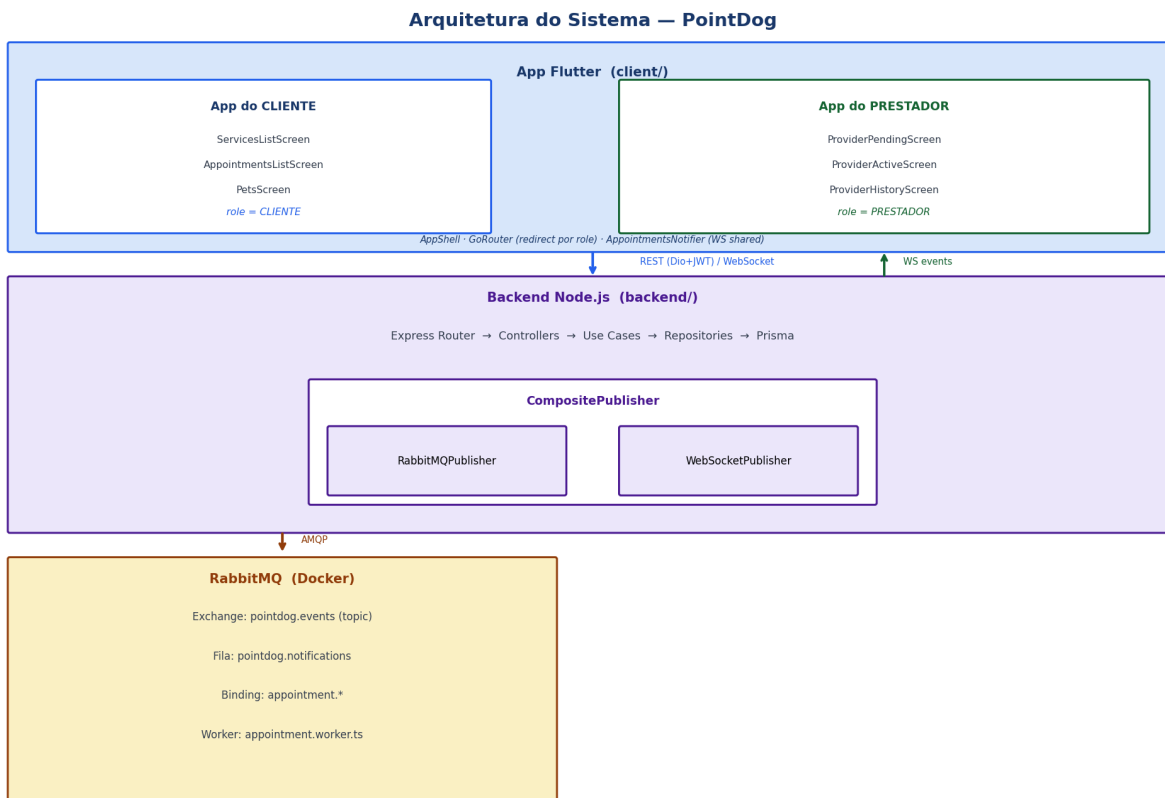


Figura 1 — Arquitetura geral do sistema PointDog

2.2 Fluxo Ponta a Ponta

O fluxo completo de um agendamento percorre todos os componentes do sistema, desde a criação pelo cliente até a notificação em tempo real ao prestador e a resposta assíncrona via RabbitMQ:

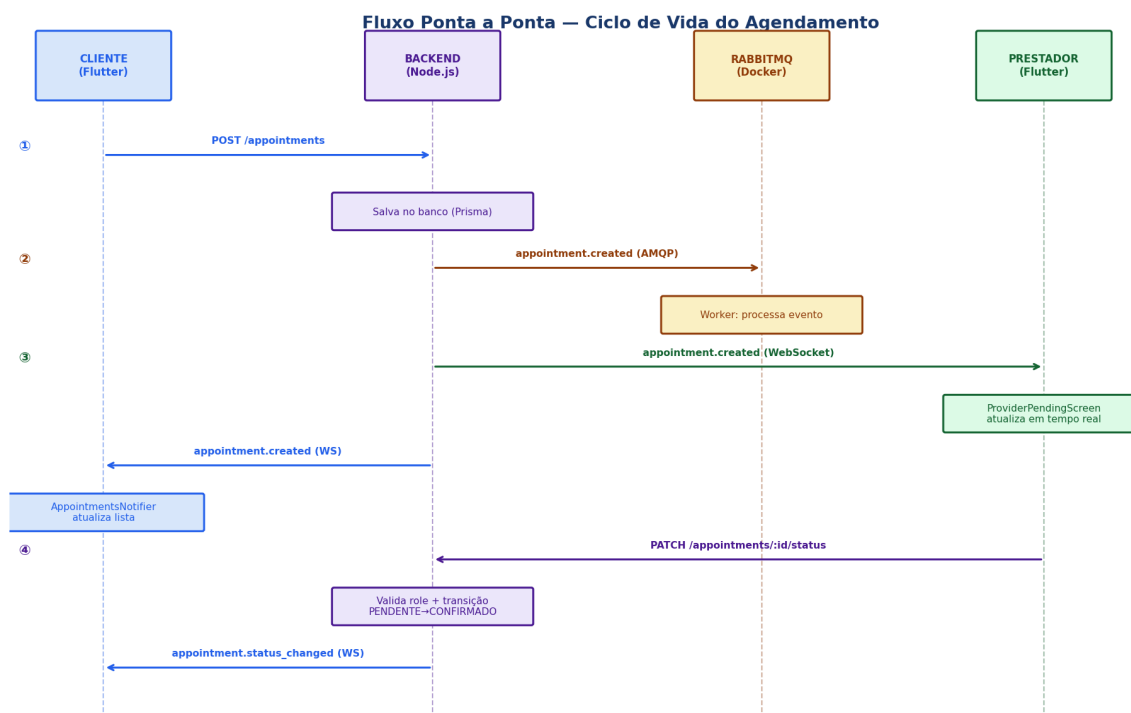


Figura 2 — Fluxo ponta a ponta de um agendamento

2.3 Arquitetura do App Flutter — Role-Based Views

A decisão central da Sprint 4 foi implementar o app do prestador dentro do mesmo projeto Flutter (client/), usando views condicionadas pela role do usuário, em vez de criar um segundo projeto separado. Essa abordagem aplica o princípio DRY (Don't Repeat Yourself): a infraestrutura de rede, os modelos e os gerenciadores de estado são compartilhados entre cliente e prestador, diferenciando apenas a camada de apresentação.

```

AuthStorage.role == 'PRESTADOR'
├── GoRouter redirect → /provider/pending
├── AppShell → bottom nav com 3 tabs do prestador
└── AppointmentsNotifier → ouve appointment.created via WS
  
```

2.4 Camadas da Arquitetura Flutter

Segundo Martin (2019), o app Flutter mantém a separação em camadas com dependências apontando apenas para dentro:

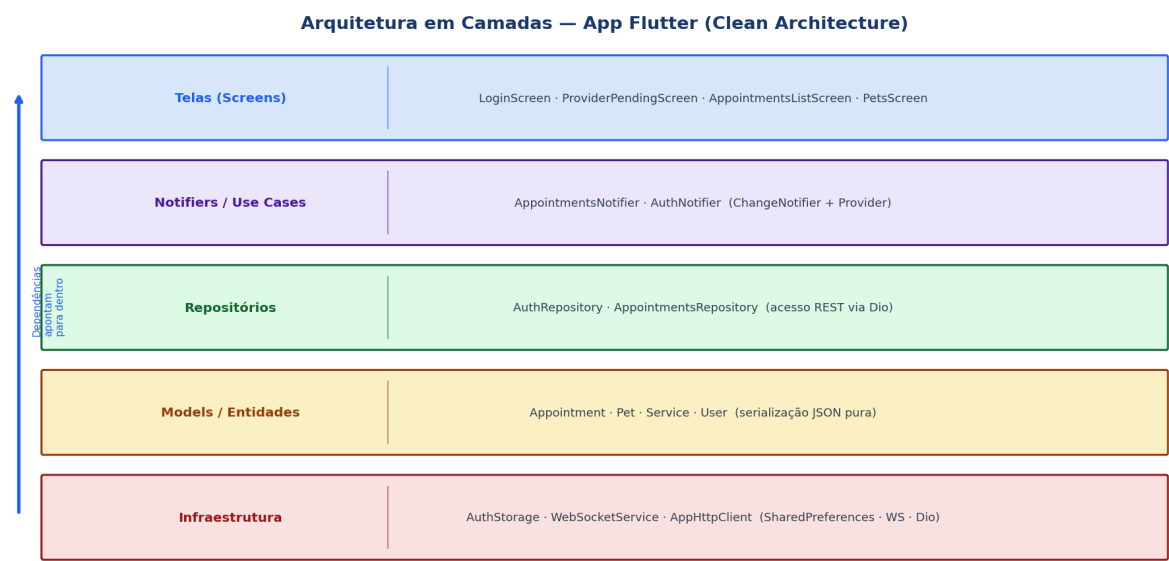


Figura 3 — Camadas da Clean Architecture no app Flutter

Camada	Componentes	Responsabilidade
Entidades (Models)	Appointment, Pet, Service, User	Estrutura de dados, serialização JSON
Repositórios	AuthRepository, AppointmentsRepository	Acesso à API REST via Dio
Notifiers (Use Cases)	AppointmentsNotifier, AuthNotifier	Lógica de estado, filtros, ações
Telas (Screens)	LoginScreen, ProviderPendingScreen, etc.	Apresentação, sem lógica de negócio
Infraestrutura	AuthStorage, WebSocketService, AppHttpClient	Persistência, rede, WebSocket

3. Decisões de Design

3.1 Roteamento WebSocket por providerId

Problema

O `WebSocketEventPublisher` original só roteava eventos para o `clientId`. O prestador nunca recebia notificação de novos agendamentos.

Solução

Refatorar o método `publish()` para extrair ambos `clientId` e `providerId` do payload, usando um `Set<string>` como destinos:

```
async publish(_routingKey: string, payload: unknown): Promise<void> {
  const event = payload as RoutableEvent
  const message = JSON.stringify(payload)
  const targets = new Set<string>()
  if (event?.payload?.clientId) targets.add(event.payload.clientId)
  if (event?.payload?.providerId) targets.add(event.payload.providerId)
  for (const userId of targets) this.sendToUser(userId, message)
}
```

Alternativa descartada: polling periódico via GET `/appointments`. Descartada por consumir mais recursos, introduzir latência artificial e desperdiçar conexões numa arquitetura que já possui WebSocket disponível.

3.2 Persistência da Role em SharedPreferences

A role (CLIENTE / PRESTADOR) é extraída da resposta do POST `/auth/login` (campo `user.role`) e persistida localmente em `SharedPreferences`, ao lado do token JWT e do `userId`. Isso permite que:

- O redirect do GoRouter aconteça sincronamente no `initState` (sem esperar o backend)
- O AppShell renderize o bottom nav correto na primeira frame

Risco: dessincronização se a role for alterada no backend sem novo login.

3.3 Recarregamento via REST no appointment.created

Quando o prestador recebe o evento `appointment.created` via WebSocket, o payload contém apenas IDs sem os nomes enriquecidos (pet, serviço). A decisão foi disparar um `loadAll()` via REST em vez de tentar montar o objeto a partir do payload:

```
if (eventType == 'appointment.created') {
  if (auth.role == 'PRESTADOR' && providerId == auth.userId) {
    loadAll(); // REST GET /appointments - retorna dados enriquecidos
  }
}
```

3.4 MOM vs. WebSocket — Divisão de Responsabilidades

O RabbitMQ e o WebSocket coexistem com responsabilidades distintas, seguindo a arquitetura de publicação/assinatura descrita por Hohpe e Woolf (2003):

Canal	Responsabilidade	Por que
-------	------------------	---------

RabbitMQ (MOM)	Processamento assíncrono de eventos de domínio	Durabilidade, desacoplamento, reprocessamento
WebSocket	Entrega em tempo real ao app	Baixa latência, conexão persistente já existente

O worker do RabbitMQ (`appointment.worker.ts`) simula notificações push (ex.: FCM para notificação fora do app). Em produção, este seria o componente responsável por enviar push notifications quando o usuário está com o app fechado.

3.5 Role-Based Views em Projeto Único

A alternativa de criar um segundo projeto Flutter para o prestador foi descartada porque geraria duplicação imediata: dois conjuntos de modelos, dois repositórios REST, dois `AppointmentsNotifier`, dois `WebSocketService`. A solução de projeto único aproveita que o Flutter separa a lógica de apresentação da lógica de negócio: um único `AppointmentsNotifier` serve tanto o cliente quanto o prestador, com a diferenciação feita pelos getters de filtro (`pending`, `active`, `history`).

4. Dificuldades Encontradas e Soluções Adotadas

4.1 Autenticação no WebSocket

Uma dificuldade que surgiu cedo foi como autenticar o usuário na conexão WebSocket. No HTTP comum, o token JWT vai no cabeçalho `Authorization`, mas a API WebSocket dos navegadores não permite definir cabeçalhos na hora de abrir a conexão. Isso inviabilizou reusar o mesmo mecanismo de autenticação das rotas REST.

A saída foi passar o token diretamente na URL: ``ws://host/ws?token=<jwt>``. No servidor, o token é extraído da query string, validado, e a conexão é associada ao ``userId`` do payload. Funciona bem para o contexto do projeto, embora em produção o ideal fosse usar tokens de vida curta para minimizar o risco de exposição do JWT na URL.

4.2 Payload do WebSocket sem dados enriquecidos

Quando o prestador recebia o evento ``appointment.created`` via WebSocket, o payload trazia apenas IDs — ``appointmentId``, ``clientId``, ``petId`` — sem os nomes de pet ou serviço. Exibir IDs brutos para o usuário não era aceitável.

A solução adotada foi, ao receber o evento, disparar uma chamada REST (``GET /appointments``) para recarregar a lista completa com todos os dados. Há um round-trip extra, mas garante que o que aparece na tela é sempre consistente com o banco. Uma alternativa seria buscar só o agendamento específico por ID, mas o ``loadAll`` foi suficiente para o escopo do projeto.

4.3 Reconexão automática do WebSocket

4.3 Reconexão automática do WebSocket

Para o escopo acadêmico, a solução foi garantir conexão estável durante a demonstração. A implementação correta exigiria um mecanismo de reconexão com tentativas espaçadas (por exemplo, aguardar 1s, depois 2s, depois 4s) — algo que Coulouris et al. (2011) apontam como prática essencial em sistemas distribuídos, onde falhas de rede são eventos esperados, não exceções.

4.4 Dificuldades nos testes

Os testes do `WebSocketEventPublisher` foram um desafio porque a classe cria um servidor WebSocket real internamente, o que tornava difícil testar o comportamento de roteamento de mensagens de forma isolada. A solução foi injetar conexões simuladas diretamente no mapa interno do publisher durante os testes, evitando a necessidade de abrir sockets reais.

Outro problema menor foi o arquivo `widget_test.dart` padrão do Flutter, que referenciava `MyApp` — o nome gerado automaticamente no `flutter create` — em vez de `PointDogApp`. Causava erro no `flutter analyze` e foi substituído por um teste mínimo que verifica apenas que a árvore de widgets inicializa sem exceções.

5. Reflexão sobre os Padrões Estudados

5.1 Event-Driven Architecture (EDA)

O projeto implementa EDA de forma prática: os use cases do backend publicam eventos de domínio (`appointment.created`, `appointment.status_changed`) sem conhecer quem os consome. Isso desacopla completamente o produtor (API REST) dos consumidores (worker RabbitMQ, WebSocket publisher).

Conforme Richardson (2018), o padrão Event-Driven é especialmente adequado quando múltiplos serviços precisam reagir ao mesmo evento sem acoplamento direto. No PointDog, o **CompositePublisher** implementa exatamente esse papel: delega para múltiplos `IEventPublisher` sem que os use cases de negócio saibam da existência de RabbitMQ ou WebSocket.

A interface **IEventPublisher** age como contrato de abstração que permite substituir ou adicionar mecanismos de entrega (ex.: FCM, e-mail) sem alterar a lógica de negócio — o **Princípio Aberto/Fechado** de Martin (2019) aplicado à camada de infraestrutura.

5.2 Message-Oriented Middleware (MOM)

O RabbitMQ com Topic Exchange permite que múltiplos consumidores se inscrevam em padrões de routing key (`appointment.*`) sem modificar o produtor. Conforme Hohpe e Woolf (2003), o Topic Exchange combina os padrões:

- Publish-Subscribe Channel: fan-out de mensagens para múltiplos consumidores
- Message Filter: o roteador determina dinamicamente quais filas recebem cada mensagem com base no padrão de routing key

A fila `pointdog.notifications` é durável e as mensagens são persistentes (`persistent: true`), implementando o padrão **Guaranteed Delivery** de Hohpe e Woolf (2003): a infraestrutura assume a responsabilidade pela entrega, liberando o produtor de implementar mecanismos de `retry`.

5.3 Clean Architecture no Backend

O backend segue os princípios de Clean Architecture (MARTIN, 2019):

- Entidades: Appointment, Pet, Service, User — independentes de frameworks
- Use Cases: CreateAppointmentUseCase, UpdateAppointmentStatusUseCase — lógica de negócio pura, sem importações de Express ou Prisma
- Adaptadores: controllers Express, repositórios Prisma, publishers de evento — traduzem entre o mundo externo e as entidades
- Frameworks: Express, Prisma, RabbitMQ, WebSocket — apenas na camada mais externa

A **Regra de Dependência** é o invariante central: as dependências de código-fonte apontam sempre para dentro (em direção às entidades), nunca para fora. Um use case pode chamar uma interface de repositório, mas nunca uma implementação Prisma diretamente. Isso permite que a mesma lógica de negócio seja testada com repositórios em memória sem nenhuma modificação.

5.4 Clean Architecture no Flutter

No app Flutter, a mesma separação é aplicada com adaptação ao ecossistema. Bailey (2023) detalha essa organização para projetos Flutter estruturados, com `models`, `repositories`, `state management` e `screens` como camadas discretas com dependências unidirecionais.

A adição das telas do prestador na Sprint 4 não exigiu modificações nas camadas de modelo ou repositório — validando empiricamente que a separação de camadas facilita extensão sem modificação (Open/Closed Principle).

5.5 REST

Os endpoints seguem as convenções REST com recursos bem definidos (`/appointments`, `/pets`, `/services`), verbos HTTP semânticos (GET, POST, PATCH, DELETE) e códigos de status apropriados (201 Created, 200 OK, 401 Unauthorized, 403 Forbidden).

O uso de `PATCH /appointments/:id/status` — em vez de `PUT` — reflete corretamente a semântica de **atualização parcial** de recurso: apenas o campo `status` é modificado, sem necessidade de enviar o recurso completo. A distinção entre `PUT` (substituição total) e `PATCH` (modificação parcial) é fundamental para a correção semântica de APIs REST.

8. Referências Bibliográfica

BAILEY, Thomas. *Flutter for beginners*. 3rd ed. Birmingham: Packt, 2023. ISBN 978-1-80323-765-8.

COULOURIS, George et al. *Distributed Systems: concepts and design*. 5th ed. Boston: Addison-Wesley, 2011. ISBN 978-0-132-14301-1.

FIELDING, Roy Thomas. *Architectural Styles and the Design of Network-based Software Architectures*. Tese de Doutorado. University of California, Irvine, 2000.

HOHPE, Gregor; WOOLF, Bobby. *Enterprise Integration Patterns*. Boston: Addison-Wesley, 2003. ISBN 978-0-321-20068-6.

MARTIN, Robert C. *Arquitetura limpa: o guia do artesão para estrutura e design de software*. Rio de Janeiro: Alta Books, 2019. ISBN 978-85-508-0608-0.

RICHARDSON, Chris. *Microservices patterns: with examples in Java*. Shelter Island: Manning, 2018. ISBN 978-1-617-29454-1.